CO - 2

EXPERIMENT NO: 1

Suppose, you often go to restaurant with friends and you have to split amount of bill. Write a program to calculate split amount of bill.

Aim

To develop a Dart program to calculate the amount each person has to pay when a restaurant bill is shared among friends.

Theory

The Bill Split Calculator is a simple application that divides the total bill amount by the number of friends. It demonstrates the use of variables, user input, arithmetic operations, and output statements in Dart. The program accepts the total bill amount and the number of friends from the user and calculates the amount each person should pay.

Algorithm

1. Start.
2. Read the total bill amount.
3. Read the number of friends.
4. Divide the bill amount by the number of friends.
5. Display the amount each person should pay.
6. Stop.

Program

```
import 'dart:io';


void main()
{
  print("===== Bill Split Calculator =====");
  stdout.write("Enter total bill amount: ");
  double billAmount = double.parse(stdin.readLineSync()!);


  stdout.write("Enter number of friends: ");
  int friends = int.parse(stdin.readLineSync()!);


  double amountPerPerson = billAmount / friends;
  print("\nEach person should pay:
₹${amountPerPerson.toStringAsFixed(2)}");
}
```

OUTPUT

```
PS C:\Users\LENOVO\bill_split> dart run
Building package executable...
Built bill_split:bill_split.
===== Bill Split Calculator =====
Enter total bill amount: 200
Enter number of friends: 5

Each person should pay: ₹40.00
```

RESULT

Thus, the Dart program to calculate the split amount of a restaurant bill was executed successfully.

EXPERIMENT NO: 2

Write a password generator in Dart. Be creative with how you generate passwords - strong passwords have a mix of lowercase letters, uppercase letters, numbers, and symbols. The passwords should be random, generating a new password every time the user asks for a new password. Include your run-time code in a main method.

Aim

To develop a Dart program that generates a random password containing uppercase letters, lowercase letters, numbers, and special characters.

Theory

A password generator is a program that creates strong and random passwords to improve security. In Dart, random passwords can be generated using the Random class and by selecting characters randomly from predefined sets of uppercase letters, lowercase letters, digits, and special symbols.

Algorithm

1. Start.
2. Define uppercase letters, lowercase letters, digits, and special symbols.
3. Combine all characters into a single string.
4. Read the required password length from the user.
5. Generate random characters using the Random class.
6. Display the generated password.
7. Stop.

Program

```
import 'dart:io';
import 'dart:math';


void main()
{
  const String chars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz01234
56789!@#\$%^&*";


  Random random = Random();


  stdout.write("Enter password length: ");
  int length = int.parse(stdin.readLineSync()!);


  String password = "";


  for (int i = 0; i < length; i++)
 {
    password += chars[random.nextInt(chars.length)];
  }
  print("\nGenerated Password: $password");
}
```

OUTPUT

```
PS C:\Users\LENOVO\password_generator> dart run
Building package executable...
Built password_generator:password_generator.
Enter password length: 8

Generated Password: 4NRmu%Uc
PS C:\Users\LENOVO\password_generator> dart run
Building package executable...
Built password_generator:password_generator.
Enter password length: 6

Generated Password: 1bk5Jr
```

Result

Thus, the Dart program to generate a random password was executed successfully.

EXPERIMENT - 3

Randomly generate a 4- digit number. Ask the user to guess a 4-digit number. For every digit the user guessed correctly in the correct place, they have a "cow". For every digit the user guessed correctly in the wrong place is a "bull." Every time the user makes a guess, tell them how many "cows" and "bulls" they have. Once the user guesses the correct number, the game is over. Keep track of the number of guesses the user makes throughout the game and tell the user at the end.

Aim

To develop a Dart program that generates a random 4-digit number and allows the user to guess it using the Cow and Bull game.

Theory

The Cow and Bull game is a number guessing game. The computer generates a random 4-digit number. The user repeatedly guesses the number. If a digit is correct and in the correct position, it is called a **Cow**. If a digit is correct but in the wrong position, it is called a **Bull**. The game continues until the user guesses the correct number.

Algorithm

1. Start.
2. Generate a random 4-digit number.
3. Read the user's guess.
4. Compare the guess with the generated number.
5. Count cows (correct digit in the correct position).
6. Count bulls (correct digit in the wrong position).
7. Display the number of cows and bulls.
8. Repeat until the user guesses the correct number.

9. Display the total number of attempts.

10. Stop.

Program

```
import 'dart:io';
import 'dart:math';

void main()
{
  Random random = Random();
  String secret = (1000 + random.nextInt(9000)).toString();
  int attempts = 0;
  print("===== Cow and Bull Game =====");

  while (true) {
    stdout.write("Enter a 4-digit number: ");
    String guess = stdin.readLineSync()!;

    if (guess.length != 4) {
      print("Please enter exactly 4 digits.");
      continue;
    }

    attempts++;
```

```
    int cows = 0;
    int bulls = 0;

    for (int i = 0; i < 4; i++) {
      if (guess[i] == secret[i]) {
        cows++;
      } else if (secret.contains(guess[i])) {
        bulls++;
      }
    }

    print("Cows: $cows");
    print("Bulls: $bulls");

    if (cows == 4) {
      print("\nCongratulations! You guessed the number.");
      print("Secret Number: $secret");
      print("Attempts: $attempts");
      break;
    }
  }
}
```

OUTPUT

```
PS C:\Users\LENOVO\cow_bull_game> dart run
Building package executable...
Built cow_bull_game:cow_bull_game.
===== Cow and Bull Game =====
Enter a 4-digit number: 1234
Cows: 0
Bulls: 0
Enter a 4-digit number: 8547
Cows: 0
Bulls: 2
Enter a 4-digit number: 6978
Cows: 0
Bulls: 2
Enter a 4-digit number: 1235
Cows: 0
Bulls: 1
Enter a 4-digit number: 7777
Cows: 0
Bulls: 0
Enter a 4-digit number: 7894
Cows: 2
Bulls: 0
```

Result

Thus, the Dart program to implement the Cow and Bull game was executed successfully.